Lecture I: Optimal Stopping

Programming: Everyday Decision-Making Algorithms

LECTURER

Dr. Nils Roemer, External Lecturer

AFFILIATION

Planningio GmbH

Today in one sentence

When options come one by one and you can’t go back, look at the first 37% without committing, then take the first one that beats them all.

WHY IT MATTERS

You visit flats in Hamburg. Each viewing, you must decide on the spot, and if you say no, the flat is gone. Choose too early and you miss better ones; wait too long and the best is behind you. Companies face the same problem when hiring, choosing suppliers or accepting offers.

Roadmap

01 The problem: choosing the best when options come one at a time

02 The rule: look first, then leap

03 Real life: when the assumptions change

Options come one by one, and there is no going back

Question: Does anybody know what optimal stopping is?

- **Optimal stopping** is the problem of choosing the **best option**
- from a **sequence of options**
- that are revealed **one by one**

Speaker notes

Topic: understanding optimal stopping problems and the famous Secretary Problem. Why this matters: optimal stopping is everywhere in life, from hiring decisions to finding apartments to choosing when to sell stocks. Today we learn the mathematical strategy that maximizes the chance of making the best choice.

Anybody an example of optimal stopping?

Flat Hunting



Hiring applicants



Dating



The Secretary Problem: hire now or keep looking?

- You are **hiring a secretary** and interview candidates **one by one**
- After each interview: **hire or continue searching**
- Once you reject a candidate, **you cannot go back**
- How to **maximize the chance of hiring the best candidate?**

Speaker notes

The name comes from the 1960s. The problem is about finding the best candidate, not about secretaries. It is also known as the marriage problem or the dowry problem.

The setup in three lines

- We have `n` candidates and interview them **one by one**
- After each one: **hire or continue searching**
- We can only **rank** candidates relative to those we have already seen (**ordinal ranking**)

Question: Where in your life can you say “this one is better than that one”, but not “this one is worth 7 out of 10”?

Speaker notes

Ordinal ranking: we can say which of two candidates is better, but we do not have a score or an absolute scale. This is what makes the problem hard, we never know how good the best candidate could be.

Two ways to fail

Question: Any idea how we could fail?

1. Reject everyone and never hire: **stopping too late**
2. Hire someone too early: **stopping too early**

How would you decide when to stop?

Roadmap

01 The problem: choosing the best when options come one at a time

02 **The rule: look first, then leap**

03 Real life: when the assumptions change

Look at the first 37%, then leap

1. **Look** at the **first 37%** of candidates without hiring anyone
2. **Remember** the best candidate so far
3. **Leap:** hire the next candidate who is **better than that one**
4. This finds the very best candidate in **37%** of all cases

Speaker notes

This is the mathematically optimal strategy. It means we still fail 63% of the time, but this is the best we can possibly do. The 37% holds for a large number of candidates; with only a few candidates, the chance is even higher.

Picking at random gets worse with every candidate

Chance of the best candidate if we **pick one at random**:

- 1 candidate: 100%, 2 candidates: 50%, 3 candidates: 33%
- 4 candidates: 25%, 5 candidates: 20%

Question: Do you see the pattern?

Pattern: 1/n. Look-and-leap **never drops below 37%**, however many candidates there are.

Speaker notes

Random choice gets worse with every extra candidate, because each individual candidate is less likely to be the best. The look-and-leap strategy does not have this problem: its success chance never drops below 37%, however many candidates there are, and with few candidates it is even higher.

Why 37%?

- Look at the first share x of candidates, then leap
- Chance of getting the best: about $-x \ln x$[1]
- In words: look longer and your benchmark improves, but the best may already be gone
- Largest at $x = 1/e \approx 37\%$[2]

1. Where does this come from? You win if the best candidate comes after your look phase and nobody before them beats your benchmark. That happens exactly when the best of all earlier candidates was in your look phase. Adding up these chances for every position, with many candidates, gives about $-x \ln x$, where x is the share of candidates you only look at.
2. To find the best look phase, we only need to know where $-x \ln x$ reaches its maximum. Setting its derivative $-\ln x - 1$ to zero gives $x = 1/e \approx 37\%$, where $e \approx 2.718$ is the base of the natural logarithm. For the full story, see Ferguson, T.S. (1989) 'Who solved the secretary problem?', Statistical Science, 4(3).

Let the computer try it out

1. Shuffle 100 candidates into a random order
2. Look at the first 37 and remember the best one
3. Hire the first candidate who beats that benchmark
4. Check: did we get the very best?
5. Repeat 10,000 times and count how often we won

The same idea in code

```python
import random
random.seed(42)

def one_round(n=100, look=37):
    candidates = list(range(n))          # n - 1 is the best
    random.shuffle(candidates)
    benchmark = max(candidates[:look])   # best of the look phase
    for c in candidates[look:]:
        if c > benchmark:                # first one who beats it
            return c == n - 1            # did we hire the best?
    return False                         # nobody beat the benchmark

wins = sum(one_round() for _ in range(10_000))
print(f"Best candidate found in {wins / 10_000:.1%} of rounds")
```

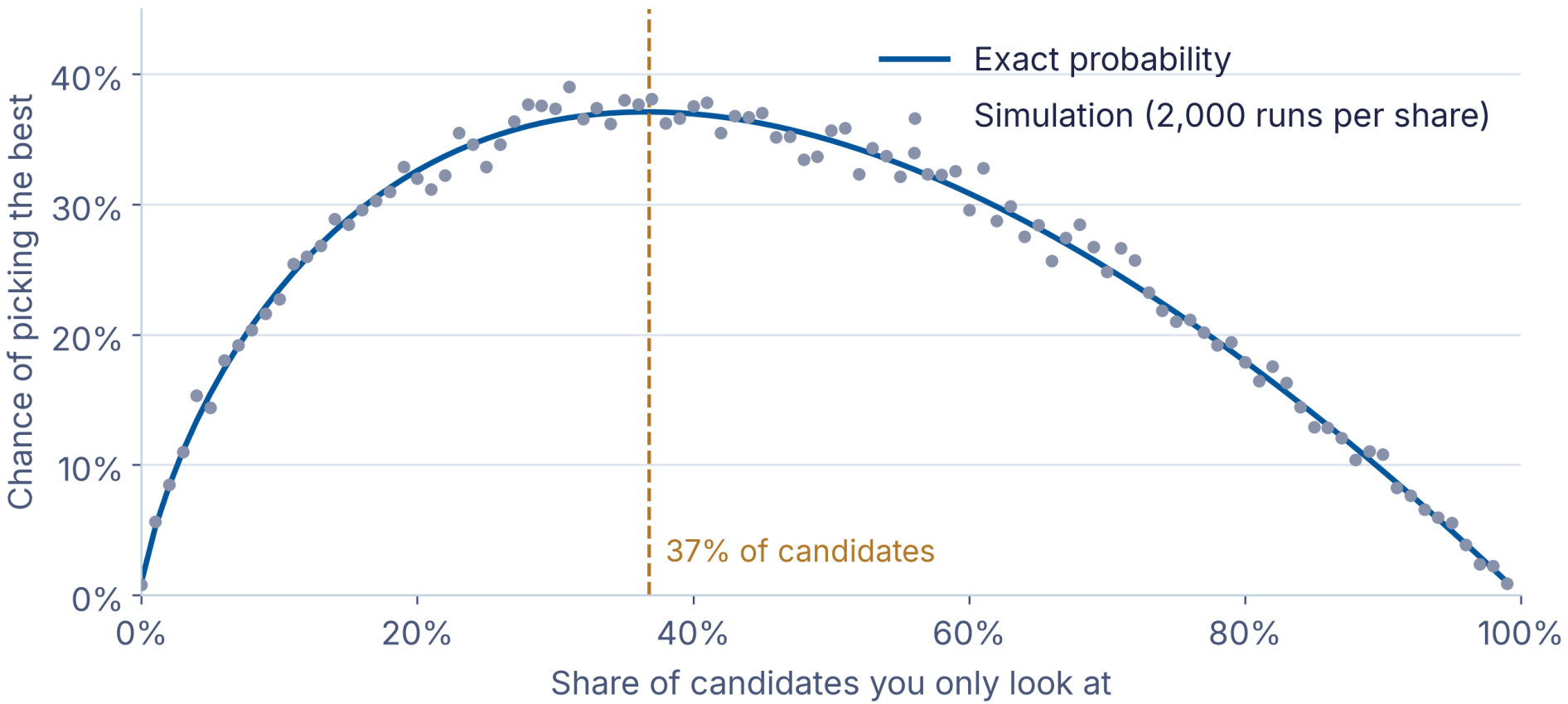
```
Best candidate found in 37.5% of rounds
```



No worries if you don’t understand every line yet. By the end of the course, you will.

Simulation: looking at 37 of 100 candidates works best

100 candidates, 2,000 simulated runs per stopping point:



Roadmap

01 The problem: choosing the best when options come one at a time

02 The rule: look first, then leap

03 **Real life: when the assumptions change**

Real life changes the assumptions

The 37% rule assumes that offers are always accepted, there is no going back, we only know rankings, and looking is free.

Question: What could be different in real life?

- The other person can **say no**
- You can **go back** to someone you passed
- You **know how good** an option is, not just how it compares
- You have **a fixed time**, not a fixed number of options
- One failure can mean **losing everything**
- Looking **costs time and effort**

If you can be rejected, leap earlier

Question: In dating, **the other person can also say no**. **When to leap?**

- 50% chance of rejection: **leap at 25%**
- **Formula:** $q^{\frac{1}{1-q}}$, q = chance that your offer is **accepted**
- In words: the more often you hear “no”, the earlier you leap

Speaker notes

Life lesson: higher risk of rejection means we should be less picky. With q = 0.5 the formula gives 0.5 to the power of 2, that is 25%. If offers are always accepted (q close to 1), the formula gives back the familiar 1/e.

Example from Brian Christian: searching from age 18 to 40, the 37% rule says get serious at 26.1. With many rejections, start at 23.5. If you can go back to earlier prospects, keep exploring until 34.4.

If you can go back, wait longer

Question: What if you could **return to someone you passed on**, but they might say no?

- Going back works only half the time: **look at 61%**, then leap
- If nobody better comes, **return to the best** you passed
- In words: a second chance makes it safe to look longer

Speaker notes

Example from the book: an immediate offer is always accepted, a belated one only with 50%. The safety net of going back shifts the look phase from 37% to 61%.

If you know the scale, use a threshold

Question: What if you **know how good** each option is, not just how it compares?

- Example: you know typing speeds, or the range of prices
- **No look phase needed:** take the first option above a threshold
- The threshold **drops** as fewer options remain
- Chance of getting the very best rises to **58%**

Speaker notes

This is the “full information” version. With absolute scores and a known distribution, you don’t need to learn a benchmark first. The threshold starts high and falls as you run out of options.

Fixed time instead of a fixed number of options

Example: one year to find a flat, viewings come as they come

Question: How should we adapt our strategy?

- **Same principle:** look for the first 37% of the time, then leap
- Now we also control **how hard we search**

Speaker notes

This holds if options arrive evenly over time. We can also decide how intensively we search, for example by viewing more flats per week. Searching harder in the look phase improves the benchmark.

The burglar: knowing when to quit

Example: each break-in succeeds with chance q, one failure means losing everything

Question: When should the burglar retire?

- Stop after about $\frac{q}{1-q}$ successes
- With $q = 90\%$: retire after **9** break-ins
- Same logic for any streak that can end badly: a winning run in the casino, a project that keeps growing in risk

In practice, people stop too early

- In about a dozen experiments, people **leap earlier** than the 37% rule suggests
- **Question:** Why might that be?
- Searching costs **time and effort**, and the rule ignores that
- If looking is expensive, leaping earlier can be **the right choice**

Speaker notes

Experiments by Seale and Rapoport (1997). The 37% rule assumes that looking is free. In real life, every viewing or interview costs time, so stopping a bit earlier is often rational, not a mistake.

The same idea in other places

- Selling a house for the **best price** (“Threshold Rule”)
- **Finding** a parking spot (“Parking Lot Problem”)

Speaker notes

Side note for drivers: an increase in occupancy from 90 to 95% doubles the search time for all drivers. Each variation changes the rule a little, but the structure stays: options come one at a time, and you cannot go back.

What we covered

THE PROBLEM: CHOOSING THE BEST WHEN OPTIONS COME ONE AT A TIME

Many decisions share one structure: options arrive one at a time, and there is no going back.

THE RULE: LOOK FIRST, THEN LEAP

Looking at the first 37% and then taking the first better option gives the best possible chance of picking the very best: 37%.

REAL LIFE: WHEN THE ASSUMPTIONS CHANGE

The rule adapts to real life: leap earlier if you can be rejected or looking is costly, wait longer if you can go back.

Why it matters

You visit flats in Hamburg. Each viewing, you must decide on the spot, and if you say no, the flat is gone. Choose too early and you miss better ones; wait too long and the best is behind you. Companies face the same problem when hiring, choosing suppliers or accepting offers.

Look first.
Then leap.

Any questions so far?

After the break: First Steps in Python

- A gentle introduction to Python in our notebooks
- We turn today's idea into **code and experiments**
- This is how **algorithmic thinking** starts: break a decision into logical steps

Speaker notes

How does optimal stopping connect to programming? Algorithmic thinking: break complex decisions into logical steps. Trade-off analysis: exploration vs. exploitation, fundamental in AI and the topic of the next lecture. Mathematical optimization: using formulas to find the best solution.

That’s it for optimal stopping! Let’s have a short break and then continue with our first Python programming session.

Literature

Interesting literature to start

- Christian, B., & Griffiths, T. (2016). Algorithms to live by: the computer science of human decisions. First international edition. New York, Henry Holt and Company.[1]

1. The main inspiration for this lecture. We have read it and discussed it in depth, always wanting to translate it into a course.

Optimal Stopping in Depth

- Ferguson, T.S. (1989). Who solved the secretary problem? Statistical Science, 4(3). doi:10.1214/ss/1177012493.

Books on Programming

- Downey, A. B. (2024). Think Python: How to think like a computer scientist (Third edition). O'Reilly. Free online
- Elter, S. (2021). Schrödinger programmiert Python: Das etwas andere Fachbuch (1. Auflage). Rheinwerk Verlag.



Think Python is free online and a great start. Schrödinger programmiert Python is a playful alternative in German.

More Literature

For more interesting literature, take a look at the literature list of this course.